Project 2 Tom & Jerry AI Agent Implementation:

Evan Litzer

I. INTRODUCTION

In my project submission, I designed a planner for Jerry, who is tasked with capturing Spike whilst evading Tom through traversal of the grid. It truly is a rock-paper-scissors situation, as a mix of both aggression and defensive avoidance must be considered and compensated for better preparation of the algorithm. My Jerry planner utilizes a combination of adversarial search, path-finding heuristics, and environmental awareness to make informed decisions for each turn. The planner's goal is to maximize Jerry's survival and capture success while staying between the memory bounds of the assignment, which proved to be a challenging task.

II. Planner Overview

A. Pursuit of Spike: A Distance Calculation*

The guidance of Jerry to his target Spike is first calculated by A* pathfinding. This is done in my code through the function a_star_length(). The method computes the shortest path length from the passed in start (tuple(Jerry)) to the goal (tuple(Spike)) using the A* searching algorithm that observes obstacles, 8-directional movement, and distance costs from the grid.

Three variables are defined: a priority queue 'q' that stores estimated cost and position tuples, a dictionary 'cost' that stores the shortest known cost from start to each visited node, and a set 'visited' that stores which nodes have been visited. Then the priority queue pushes Jerry's current node and its cost is set to 0. After that, the while loop iterates under the condition that q is not empty. In the loop, the first node in 'q' is popped and the current node is first checked to be the goal. If it is, then its cost is returned.

Otherwise, all directions from the current node are observed and their nodes are checked for validity. Then, its cost is computed through current cost added by 1. If the node hasn't been visited yet--or if the cost of the node is lower than the currently stored value, the cost dictionary and visited set is updated with the new node information. Then the A* scoring formula is computed, setting the priority cost value to the accumulated cost of the newly discovered node added to the Euclidean distance from the node to the goal position. Then the node and the priority cost value are pushed into the priority queue, and the process repeats with either more neighbors or further popping of the priority queue.

The loop runs until the priority queue is empty and returns infinity or the cost to get to the goal. The method is called in the evaluate function to calculate the distance_to_spike variable, which is weighted by constant -1 and returned by evaluate.

B. Evasion from Tom: Weighted Manhattan Distance

Jerry must also avoid being caught by Tom to consistently have a score greater than 0. To model this, the distance_from_tom variable is calculated through the Manhattan distance calculation between Tom and Jerry's positions. This occurs in the evaluate function, and its determined value is multiplied by a constant of 2 in its return statement. Originally I had an A* calculation instead of this simple formula, but it consumed too much memory.

C. Danger Penalty & Mobility Bonus

To discourage risky moves without completely restricting them, the planner applies an inverse square formula based on the proximity to Tom's position. The formula is as follows:

danger_penalty = 10 / ((dist_from_tom + 1)^2)

This formula will imply a strong penalty when Jerry is near Tom's position, but will decrease exponentially as distance between the two characters increase. A constant of 3 is multiplied with the danger_penalty and is included in the evaluate return statement.

The mobility bonus encourages Jerry to avoid traps and provide positions with more readily available neighboring nodes. This is designed for Jerry to avoid corners and bottlenecks brought along by obstacles. The bonus formula is as follows:

mobility = sum(valid(jerry+d, world) for d in DIRECTIONS).

This sums all the valid nodes around Jerry's position. The calculated value is then multiplied by a constant of 0.5 and is included in the evaluate return statement.

D. Adversarial Search

The planner uses an adversarial decision-making process based on alpha-beta pruning as described in the lectures. This essentially allows Jerry to look ahead two moves, which includes his own move and the best response from Tom. It also prunes suboptimal branches to reduce computation time.

This is executed through the alpha_beta function, which is called in the best_move function. The alpha_beta method first checks if the passed in depth is set to 0, in which it returns the

evaluate function of the current state representing the heuristic of the search tree leaf node.

If the passed in boolean ‘maximizing’ is set to true, then it is hypothetically Jerry’s turn. All possible moves for Jerry are tried, and recursion is used to simulate Tom’s best counter-move with ‘maximizing’ then set to false. The highest score is stored through the alpha variable, and the algorithm prunes the branch if it cannot do better than beta, represented by the break statement.

If the maximizing boolean variable is set to false, then it is Tom’s hypothetical turn. The loop tries all of Tom’s valid moves and uses recursion to simulate Jerry’s next move. While doing this, it tracks the lowest score, which is represented by beta. It prunes a branch if it is worse than what Jerry can already achieve (alpha).

This is minimaxing and max-minning. When it is Jerry’s turn, he is the maximizing player and assumes Tom will act to minimize his (Jerry’s) score. The vice-versa is true for Tom’s turn. The alpha_beta function will return the minimum or maximum evaluation depending on the maximizing boolean.

E. Putting it Together

The planner is constructed within the PlannerAgent class, which contains two different methods. The first one is the constructor, which currently sets the depth of the planner to 2. Any increase or decrease in depth will result in either too long of runtime or decreased agent performance.

The second method is plan_action. This function calls best_move(), which iterates through every possible direction Jerry can possibly take. After validity checks, it uses alpha_beta() to simulate both Jerry’s action and Tom’s optimal response up to the depth value.

Every leaf state in the search tree is scored using the evaluate() function. It combines several factors including the A* path length to the goal (Spike), Manhattan distance from the enemy (Tom), an inverse-square penalty for nearness to Tom, and a mobility bonus for increased validity of future moves. Each of these values have constants attached to them representing their importance to move choice. This planner is certainly on the conservatively defensive side of submissions based on the values of constants.

The constants have been fine tuned through constant training of my algorithm. I have come to find that staying aggressive while also accounting for distance from enemy, danger penalty, and mobility equally has been the best strategy. That is why the constants for the planner are (-3, 1.35, 1.5, 1.4).

Once every valid option has been observed and simulated, the planner selects the move that maximizes his evaluation score. This evaluation score greatly considers Tom’s presence, even more so than Jerry’s pathing to Spike. This approach allows the planner to prioritize staying alive while still staying in pursuit of the goal, to capture Spike!